ORM & Hibernate — Day 1 Lab Pack

Labs covering the full Day 1 session: JDBC/ORM concepts, Hibernate fundamentals, and JPA

LAB 1

Hibernate Native API — Employee CRUD

Objective

Map the `Employee` entity with JPA annotations, configure Hibernate's native `SessionFactory`, and implement full CRUD (Create, Read, Update, Delete) using `Session` — reinforcing dirty checking, the entity lifecycle, and the first-level cache covered in lecture.

Requirements

Item	Requirement
Maven dependencies	hibernate-core 6.4.4.Final, com.h2database:h2 2.2.224
Config file	hibernate.cfg.xml (native Hibernate configuration, in src/main/resources)
Entity class	Employee — mapped with @Entity, @Table, @Id, @GeneratedValue, @Column
Fields	id (int, PK, IDENTITY strategy), name (String), department (String), salary (double)
Access class	EmployeeDao (or similar) exposing create/read/update/delete methods
Entry point	A Main class demonstrating each CRUD operation, with console output

Starter — hibernate.cfg.xml skeleton

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">org.h2.Driver</property>
    <property name="hibernate.connection.url">jdbc:h2:mem:empdb</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <mapping class="com.training.model.Employee"/>
  </session-factory>
</hibernate-configuration>
```

Tasks

Part A — Mapping

1. Create the `Employee` entity exactly as shown in the lecture slides (`@Entity`, `@Table(name="employee")`, `@Id` with `GenerationType.IDENTITY`, and `@Column` on every field).
2. Add the no-argument constructor required by Hibernate, plus a convenience constructor (`name, department, salary`).
3. Add getters/setters for every field, and a readable `toString()`.

Part B — SessionFactory bootstrap

1. Write a `HibernateUtil` class that builds a `SessionFactory` once (static field) from `hibernate.cfg.xml`, and exposes a `getSessionFactory()` accessor.
2. Confirm the factory builds without exceptions before moving on — run a smoke test that just opens and immediately closes a `Session`.

Part C — CRUD operations

1. **Create:** implement `save(Employee e)` using `session.persist(e)` inside a transaction. Insert at least three employees.
2. **Read (single):** implement `findById(int id)` using `session.get(Employee.class, id)`.
3. **Read (all):** implement `findAll()` using an HQL query (`session.createQuery("from Employee", Employee.class)`).
4. **Update:** implement `giveRaise(int id, double amount)` that loads the employee, mutates the salary field, and commits — without calling any explicit update method. Print the salary before and after to confirm dirty checking fired.
5. **Delete:** implement `delete(int id)` using `session.remove(...)`.
6. In your `Main` class, call each method in order (create → read → update → delete → read again) and observe the generated SQL in the console (`hibernate.show_sql` is already on).

Part D — Entity Lifecycle Walkthrough

1. Right after `new Employee(...)` but before calling `persist()`, print the object and note its id is unset — this is the **Transient** state (Hibernate has no knowledge of it yet).
2. After `session.persist(e)`, print the object again — it is now **Persistent/Managed**, tracked by the Session.
3. After `session.get(...)` loads the same row again in the same `Session`, print whether the two references are == equal — confirming it's the same managed instance, not a new copy.
4. Close the `Session`, then mutate a field on the now-**Detached** object. Open a brand-new `Session` and re-load the same row by id — confirm the mutation was **not** saved.
5. Call `session.remove(...)` on a managed entity and commit — confirm via a fresh `session.get(...)` that the row is gone (**Removed** state).
6. In one sentence per state, write what distinguishes Transient / Persistent / Detached / Removed — referencing the lifecycle diagram from lecture.

Part E — First-Level (Session) Cache

1. Within a single `Session`, call `session.get(Employee.class, id)` twice for the **same id**. Print whether the two returned references are == equal.
2. Watch the console SQL log (`hibernate.show_sql`): confirm only **one** SELECT was issued for the two `get()` calls — the second one was served from the first-level cache.
3. Close that `Session` and open a new one, then repeat the same `get()` call for the same id. Confirm a **new** SELECT fires — proving the first-level cache is scoped to a single `Session`, not shared across sessions.

Part F — Group Discussion

1. What would the update step in Part C have required in raw JDBC (think: building an UPDATE statement, binding parameters, executing it), and what did Hibernate save you from writing?

Deliverables

- `Employee.java`, `HibernateUtil.java`, `EmployeeDao.java`, `Main.java`
- Console output (screenshot or pasted log) showing the generated SQL for each CRUD operation, the lifecycle walkthrough (Part D), and the cache demonstration (Part E)
- One sentence per lifecycle state (Part D, task 6)
- One short paragraph answering the Part F discussion question

> **Stretch goal (optional)** — Add a second entity, `Department`, with no relationship mapping yet — just a standalone table. Query it independently to confirm multiple mapped classes can coexist in one `SessionFactory`. (Relationships between entities are covered on Day 2.)

LAB 2

JPA Configuration & Hello JPA

Objective

Reconfigure the same `Employee` mapping under the standard JPA API (`persistence.xml`, `EntityManagerFactory`, `EntityManager`) and reproduce the same CRUD behavior from Lab 1 — confirming that the underlying concepts transfer directly to the vendor-neutral API.

Requirements

Item	Requirement
Maven dependencies	Same as Lab 1 (hibernate-core acts as the JPA provider)
Config file	persistence.xml (in src/main/resources/META-INF/), NOT hibernate.cfg.xml
Persistence unit name	empPU, transaction-type="RESOURCE_LOCAL"
Entity class	Reuse the same Employee class from Lab 1 — no changes needed
Access class	EmployeeJpaDao (or similar) using EntityManager instead of Session
Entry point	A Main class mirroring Lab 1's flow, using standard JPA calls

Starter — persistence.xml skeleton

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence" version="3.0">
  <persistence-unit name="empPU" transaction-type="RESOURCE_LOCAL">
    <class>com.training.model.Employee</class>
    <properties>
      <property name="jakarta.persistence.jdbc.url" value="jdbc:h2:mem:empdb"/>
      <property name="jakarta.persistence.jdbc.driver" value="org.h2.Driver"/>
      <property name="hibernate.hbm2ddl.auto" value="update"/>
      <property name="hibernate.show_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

Tasks

Part A — Configuration

1. Create `persistence.xml` under `src/main/resources/META-INF/` — the location is mandatory, JPA will not find it elsewhere.
2. Register the same `Employee` class from Lab 1 under `<class>` — do not create a second entity class.
3. Bootstrap an `EntityManagerFactory` with `Persistence.createEntityManagerFactory("empPU")` and confirm it builds without exceptions.

Part B — Hello JPA (minimal example)

1. Before building the full DAO, reproduce the lecture's "Hello JPA" example directly in `Main`: create one `Employee`, `em.persist(...)` it, commit, then `em.find(...)` it back by id and print its name.

2. Confirm this runs end-to-end against H2 before moving to the fuller CRUD version below.

Part C — CRUD via EntityManager

1. **Create:** implement using `em.persist(e)` inside `em.getTransaction().begin()/commit()`.
2. **Read (single):** implement using `em.find(Employee.class, id)`.
3. **Read (all):** implement using JPQL — `em.createQuery("SELECT e FROM Employee e", Employee.class)`.
4. **Update:** load an employee, mutate the salary, commit — confirm dirty checking behaves identically to Lab 1.
5. **Delete:** implement using `em.remove(...)`.
6. Run the same create → read → update → delete → read sequence as Lab 1 and compare the console SQL output side by side.

Part D — Comparison

1. Fill in the comparison table below (copy it into your submission) after running both labs.

Operation	Hibernate Native (Lab 1) / JPA (Lab 2)
Bootstrap class	SessionFactory / EntityManagerFactory
Unit-of-work class	Session / EntityManager
Config file	hibernate.cfg.xml / persistence.xml
Create	session.persist(e) / em.persist(e)
Read by id	session.get(...) / em.find(...)
Query language	HQL / JPQL
Delete	session.remove(...) / em.remove(...)

Deliverables

- `persistence.xml`, `EmployeeJpaDao.java`, `Main.java` (including the standalone Hello JPA snippet from Part B)
- Completed comparison table from Part D
- One short paragraph answering: *"If your team decided to switch from Hibernate to EclipseLink next year, what would have to change in this lab, and what would not?"*

> **Stretch goal (optional)** — Try calling `session.get(...)` (Lab 1 style) against an entity that was loaded through an `EntityManager` that has since been closed. Confirm you get a detached-entity style problem here too — the lifecycle rules are identical across both APIs.

Submission Checklist

- Lab 0 written answers (discussion questions)
- All source files for Labs 1 and 2, organized by package

- Console output/logs demonstrating each CRUD operation, the lifecycle walkthrough, and the cache demonstration actually ran
- Lab 1 Part D lifecycle sentences, and the Part F discussion paragraph
- Lab 2 Part D comparison table, and its closing discussion paragraph

---

Coming up on Day 2: entity associations (@OneToMany, @ManyToOne, @OneToOne, @ManyToMany), unidirectional vs. bidirectional relationships, and HQL/JPQL querying.